2. Robin Hood Hashing - Pedro Celis (1986)

actual position

Minimize probe distance $d(x) = (g(x) - h(x))$ %TableSize

Reduce variance in look-up cost of linear probing by robbing slots from rich keys (large $d(x)$) and give to poor keys (small $d(x)$) .

acos atoi char define exp
ceil cos float

bucket	x	$d(x)$
0	acos	0
1	atoi	1
2	char	0
3	ceil	1
4	cos	2
5	define	2
6	exp	2
7	float	2
8		
9		
10		
… …		
25		

- 👍 Same asymptotic growth as linear probing while having lightly lower expected probes.
- 👎 Slow insertion and complex deletion
- 👎 Requires storing probe distance

Provide significant improvement in worst-case behavior from O(*N*) to O(log *N*). Predictable performance with high load factor tolerance.

```
void  RobinHood_Insert ( ElementType Key, HashTable H )
{
   Position Pos = Hash( Key, H->TableSize );
   int  CurrentDisp = 0;
   ElementType currentKey = Key;
   while( true ){
          if( H->TheCells[Pos ].Info != Legitimate) {/* Empty or deleted*/
                H->TheCells[ Pos ].Info = Legitimate;
                H->TheCells[ Pos ].Element = currentKey;
                H->TheCells[ Pos ].Disp = CurrentDisp;
                return true;
           }
           if(CurrentDisp > H->TheCells[ Pos ].Disp ) {/* Rob the rich*/
                 swap(&currentKey, &H->TheCells[ Pos ].Element);
                 swap(&CurrentDisp, &H->TheCells[ Pos ].Disp );
           }
           Pos = (Pos+1) % H->TableSize ;
           CurrentDisp++;
   }
}
```